In [3]:
```
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Flatten, Dense
```

In [4]:
```
data = pd.read_csv("cifar-10.csv")

print(data.shape)
print(data.head())
print(data.columns)
```

```
(50000, 3073)
   pixel_0  pixel_1  pixel_2  pixel_3  pixel_4  pixel_5  pixel_6  pixel_7  \
0       59       43       50       68       98      119      139      145
1      154      126      105      102      125      155      172      180
2      255      253      253      253      253      253      253      253
3       28       37       38       42       44       40       40       24
4      170      168      177      183      181      177      181      184

   pixel_8  pixel_9  ...  pixel_3063  pixel_3064  pixel_3065  pixel_3066  \
0      149      149  ...          58          65          59          46
1      142      111  ...          42          67         101         122
2      253      253  ...          83          80          69          66
3       32       43  ...          39          59          42          44
4      189      189  ...          88          85          82          83

   pixel_3067  pixel_3068  pixel_3069  pixel_3070  pixel_3071  label
0          57         104         140          84          72      6
1         133         136         139         142         144      9
2          72          79          83          83          84      9
3          48          38          28          37          46      4
4          79          78          82          78          80      1

[5 rows x 3073 columns]
Index(['pixel_0', 'pixel_1', 'pixel_2', 'pixel_3', 'pixel_4', 'pixel_5',
       'pixel_6', 'pixel_7', 'pixel_8', 'pixel_9',
       ...
       'pixel_3063', 'pixel_3064', 'pixel_3065', 'pixel_3066', 'pixel_3067',
       'pixel_3068', 'pixel_3069', 'pixel_3070', 'pixel_3071', 'label'],
      dtype='object', length=3073)
```

In [5]:
```
X = data.drop("label", axis=1).values
y = data["label"].values
```

In [6]:
```
X = X / 255.0
```

In [7]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42, stratify=y
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (45000, 3072)
Testing data: (5000, 3072)
```

In [12]:
```
model = Sequential([
    Dense(512, activation="relu", input_shape=(3072,)),
    Dense(10, activation="softmax")
])

model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 512)	1,573,376
dense_9 (Dense)	(None, 10)	5,130

Total params: 1,578,506 (6.02 MB)

Trainable params: 1,578,506 (6.02 MB)

Non-trainable params: 0 (0.00 B)

In [13]:
```
model.compile(
    optimizer="sgd",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
)
```

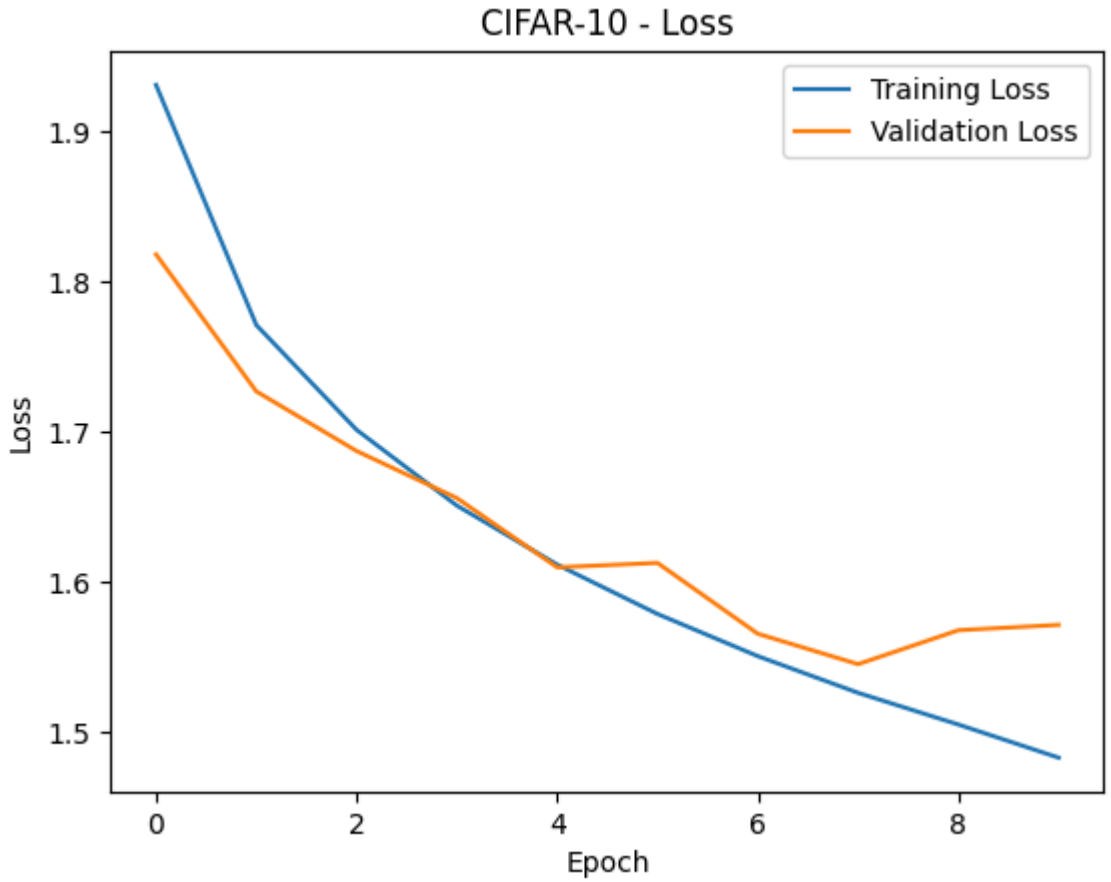
```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 13s 19ms/step - accuracy: 0.3103 - loss: 1.9311 - val_accuracy: 0.3698 - val_loss: 1.8182
Epoch 2/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.3772 - loss: 1.7711 - val_accuracy: 0.4031 - val_loss: 1.7269
Epoch 3/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 11s 17ms/step - accuracy: 0.4049 - loss: 1.7012 - val_accuracy: 0.4107 - val_loss: 1.6871
Epoch 4/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.4221 - loss: 1.6507 - val_accuracy: 0.4276 - val_loss: 1.6557
Epoch 5/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 19s 14ms/step - accuracy: 0.4372 - loss: 1.6114 - val_accuracy: 0.4407 - val_loss: 1.6096
Epoch 6/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 11s 16ms/step - accuracy: 0.4489 - loss: 1.5785 - val_accuracy: 0.4367 - val_loss: 1.6125
Epoch 7/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 10s 15ms/step - accuracy: 0.4602 - loss: 1.5504 - val_accuracy: 0.4544 - val_loss: 1.5653
Epoch 8/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 11s 18ms/step - accuracy: 0.4707 - loss: 1.5259 - val_accuracy: 0.4704 - val_loss: 1.5450
Epoch 9/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 12s 19ms/step - accuracy: 0.4769 - loss: 1.5048 - val_accuracy: 0.4544 - val_loss: 1.5677
Epoch 10/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 8s 13ms/step - accuracy: 0.4838 - loss: 1.4827 - val_accuracy: 0.4393 - val_loss: 1.5712
```

In [14]:
```
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

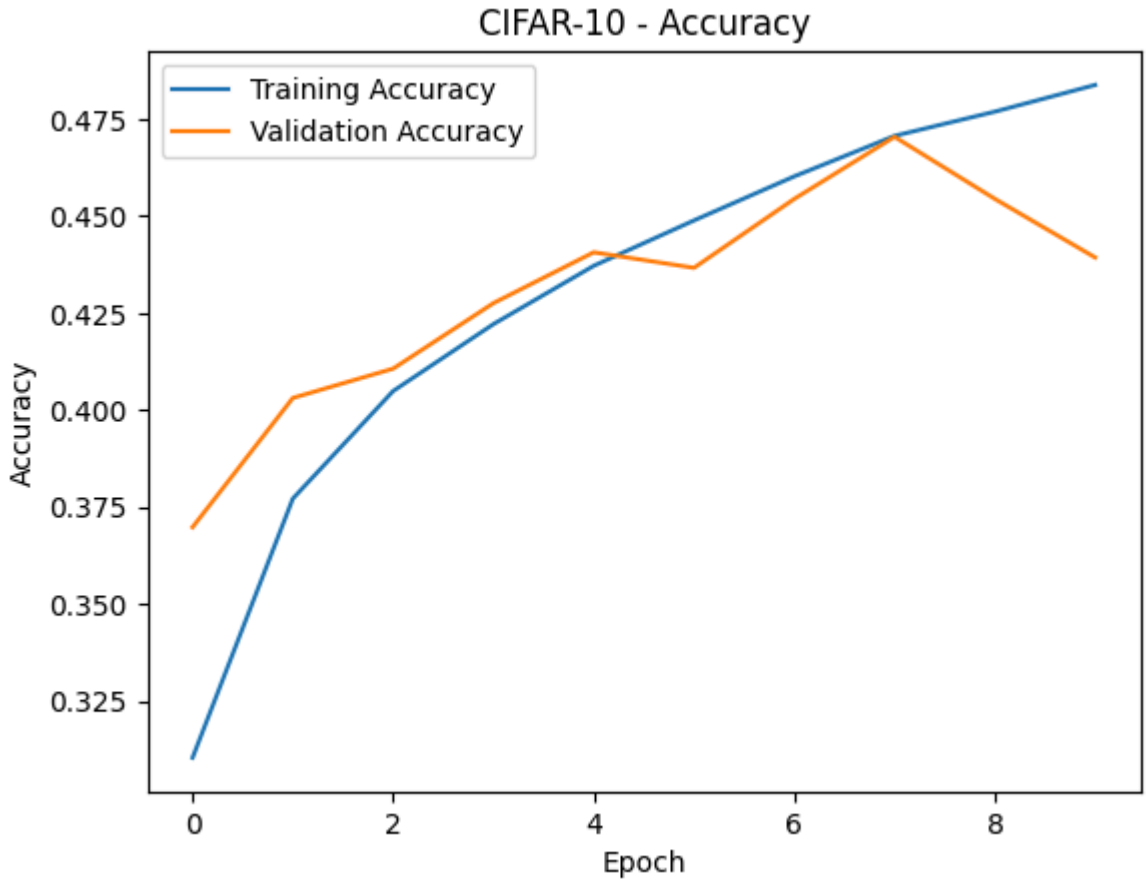
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
Test Loss: 1.5671054124832153
Test Accuracy: 0.4438000023365021
```

In [15]:
```
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("CIFAR-10 - Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



In [16]:
```
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("CIFAR-10 - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


CIFAR-10 - Accuracy
Training Accuracy
Validation Accuracy
Accuracy
0.475
0.450
0.425
0.400
0.375
0.350
0.325
0
2
4
6
8
Epoch

In [ ]: